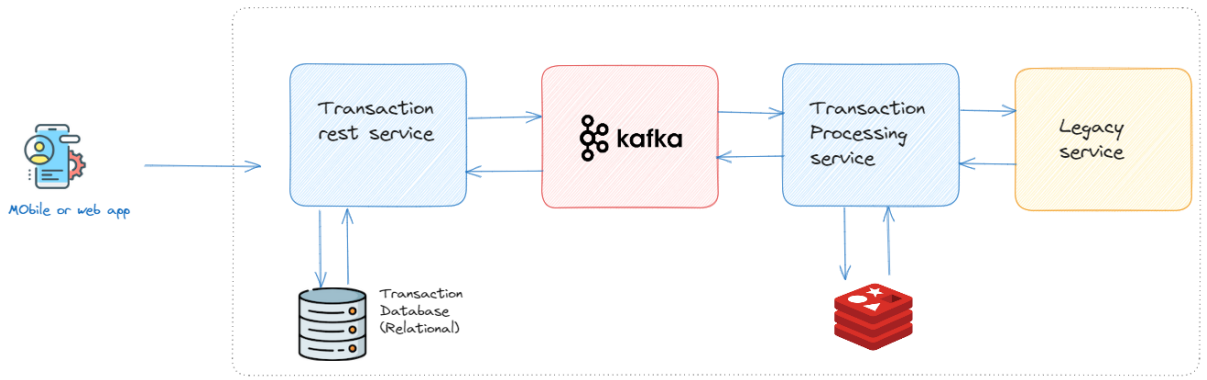
Introduction

This document outlines the architecture and design of a scalable REST API for a banking system that handles debit and credit transactions for individual accounts. The goal is to create a system that efficiently manages a large volume of requests in parallel without overwhelming an existing legacy system with limited throughput capabilities. Key concepts employed in this solution include:

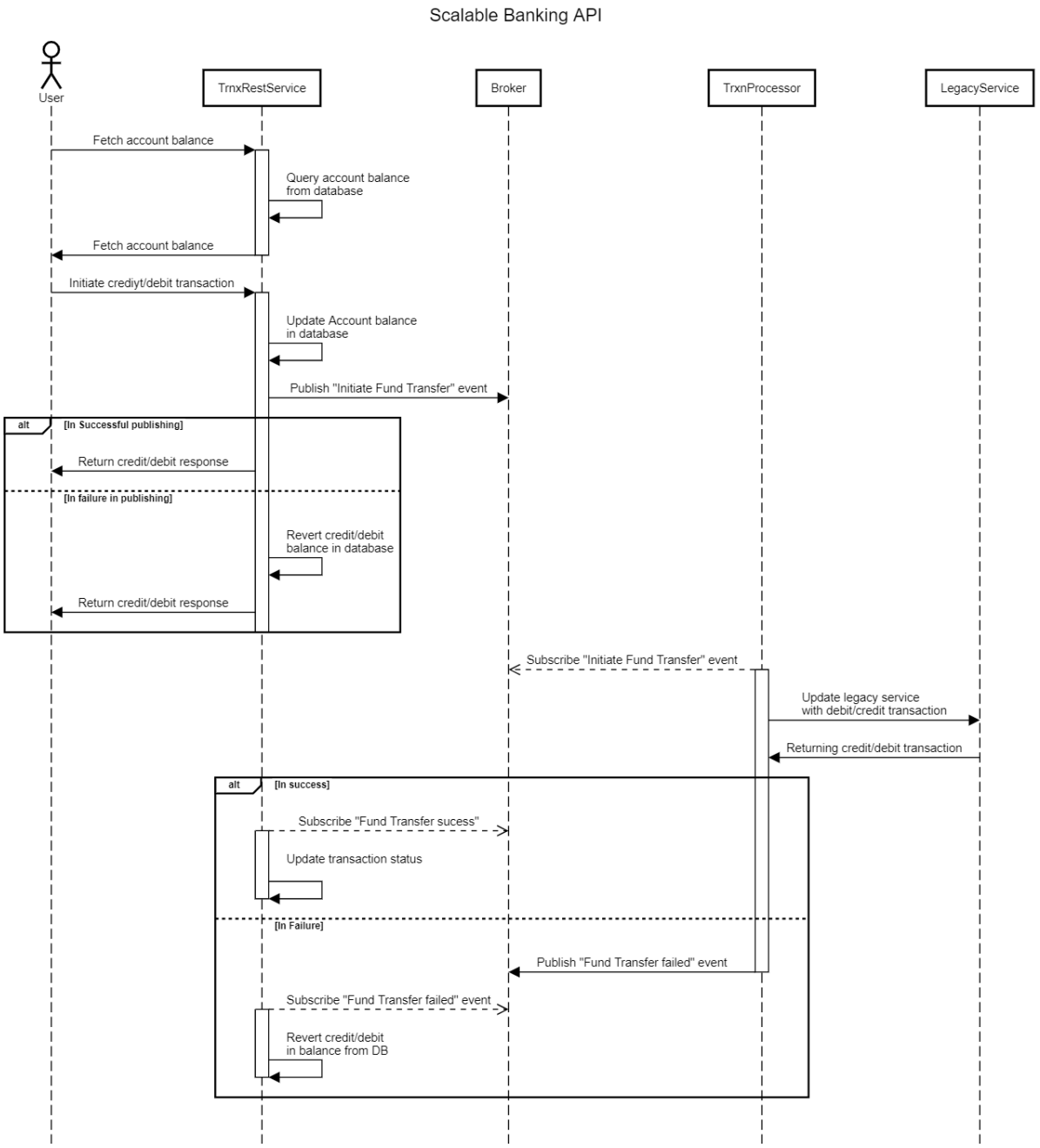
- **CQRS** (Command Query Responsibility Segregation): Separation of write (transaction) and read (balance) operations.
- **Event-Driven Architecture**: Microservices communicate via events for scalability and maintainability.
- **Microservices**: Breaking down the system into smaller, independent services.
- **Kafka**: Facilitating communication between microservices.

Block diagram :



Sequence Diagram :

Please find the link



MicroService Components :

The different Components in Microservice ecoSystem Are as given below,

Transaction-Rest-Service

- Functionality:
  - This service handles debit and credit transactions by exposing the REST API for transactions and balance API.

        - It saves the balance information in the relational database to provide more real time balance.
        - Also publishes an “Initiate Fund Transfer” event to kafka which will eventually update the legacy system asynchronously.
    - Technology: Java and Spring Boot.(For demonstration, using CompletableFuture for demoing non blocking API)
    - Connecting to : Database, Kafka
    - Database: Relational database (e.g., H2 for demo, PostgreSQL for production).
    - Kafka Interaction:
        - Publishes an event to kafka topic “Initiate Fund Transfer”.
        - Consumes events from the Kafka topic “Fund Transfer failure” .

Transaction-Processor-Service

- Functionality:
    - It subscribed to the “Initiate Fund Transfer” event with kafka.
    - Once this event is received, It will update the legacy system through rest api.
    - If the Call to the legacy system is success no actions take, else it publishes an “Fund Transfer failure” events to broker
    - It communicates with the Legacy-Service to ensure transaction processing without overwhelming it.
- Technology: Java and Spring Boot.
- Kafka Interaction:
    - Consumes an event to kafka topic “Initiate Fund Transfer”.
    - Publishes events from the Kafka topic “Fund Transfer failure” .

Kafka-Server

- Functionality: Kafka serves as a message broker for facilitating communication between microservices through an event-driven architecture.
- Technology: Development using Spring Kafka (We can use apache kafka).

Legacy-Service

- Functionality:
    - The Legacy-Service serves as a mock legacy system simulator, emulating the behaviour and limitations of the actual legacy system.
    - It exposes an API for receiving transaction requests, which will be used by the Transaction-Processor-Service.
- Technology: Spring Boot for building REST APIs. (Restrict spring boot tomcat configuration to stimulate the legacy API)

Case Study on Relational Database

The choice of a relational database for the Legacy-Service is based on the need for data consistency and transactional integrity in a banking system. Here's why:

- Data Consistency: Banking systems require strong data consistency to ensure that every transaction is processed accurately and reliably. Relational databases enforce data integrity constraints, ensuring that the data is always in a consistent state.

- Transactional Integrity: In banking, transactions often involve complex operations that must be performed atomically. Relational databases support ACID (Atomicity, Consistency, Isolation, Durability) properties, guaranteeing that transactions are executed reliably and ensuring that the database remains in a consistent state even in the face of system failures.
- Data Relationships: Banking data often involves complex relationships between entities (e.g., customers, accounts, transactions). Relational databases are well-suited for modeling and querying such relationships.
- Regulatory Compliance: The banking industry is subject to strict regulatory requirements. Relational databases provide robust audit trails and reporting capabilities, making it easier to comply with regulatory standards.

Case Study on Event-Driven Architecture
The decision to opt for an event-driven architecture across the entire microservices system is driven by several considerations:

- Scalability: Event-driven architectures enable horizontal scalability, allowing each microservice to scale independently based on its specific workload. This is crucial for handling a large volume of parallel requests without overburdening any single component.
- Loose Coupling: Event-driven communication fosters loose coupling between microservices. Each service is independent and does not rely on the synchronous availability of others. This decoupling improves system resilience and agility.
- Asynchronous Processing: Events allow for asynchronous processing, which means that microservices can handle tasks independently and in parallel. This is essential for efficiently managing high-throughput scenarios without creating bottlenecks.
- Fault Tolerance: Event-driven architectures promote fault tolerance. If one microservice experiences issues, others can continue processing events, ensuring system availability and reliability.
- Real-Time Responsiveness: By using events, the system can provide real-time updates and information while maintaining scalability and responsiveness.

In summary, an event-driven architecture is chosen to ensure that the scalable microservices can efficiently handle a large volume of parallel requests, maintain data consistency, and provide real-time responsiveness while remaining resilient and loosely coupled. This approach aligns with best practices for building robust and scalable banking systems.